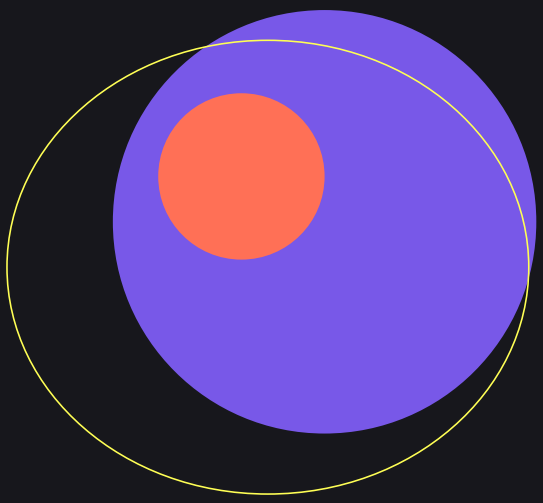


VISHNU / LEARNING JOURNAL / 01

# From Angular starter to portfolio.

A practical record of what we have built, what the Angular pieces mean, how to talk about the work in an interview, and what to practise next.

| | |
|---|---|
| PROJECT | PERSONAL PORTFOLIO |
| FRAMEWORK | ANGULAR 22 · STANDALONE · SSR SCAFFOLD |
| CURRENT MILESTONE | HOME PAGE ROUTE RENDERS LOCALLY |
| SNAPSHOT | 02 NOVEMBER 2026 |

## A NOTE ON SCOPE

This guide separates implemented work from ideas we have only planned. The Angular portfolio is at its first working route; the backend, blog, admin, and Angular Material are future learning milestones.

# 01 / Learning notes

The project goal, the decisions made, and the exact state of the Angular implementation.

## What the portfolio is for

The site is intended to help Vishnu present his experience and skills to recruiters and hiring managers, while also showing selected projects to potential clients. The main narrative is professional experience and capability; the supporting sections are projects, a visual/text-led About story, and contact. A blog and an admin area are planned for later.

## How the work evolved

- Started with the portfolio concept and a content outline focused on career outcomes, case studies, About storytelling, and clear contact routes.
- Built a standalone HTML/CSS/JavaScript visual prototype, iterated the art direction, added a theme switch and animation, and kept it at the repository root as a reference. It is not the Angular implementation.
- Created project documentation: architecture, content/API proposal, content authoring checklist, development workflow, production checklist, roadmap, and an Angular/SSR decision record.
- Set up the Angular CLI scaffold with standalone components and SSR support. Installed dependencies and ran the development server.
- Created the HomePage component, registered it at the empty route, and placed a router outlet in the root component template.
- Resolved NG05104 by restoring the app-root host element in src/index.html. The root component now mounts and the HomePage placeholder renders locally.

## Current project state

| AREA | STATUS |
|---|---|
| Angular setup | Angular CLI 22.2.2 scaffold; Node 24.21.0 and npm 11.14.1 were available. Git 2.56.0 was added to the user PATH. |
| Home route | Implemented. The root route displays the generated HomePage component in localhost. |
| Portfolio sections / API / blog / admin | Not yet implemented. They are planned. The backend and hosting are not selected. |
| Git baseline commit | Git was initialized. The conversation did not confirm whether the first commit was completed. |

**Be precise in an interview: this is an actively developed Angular learning project, not yet a deployed, backend-powered portfolio.**

# 02 / Read the Angular files

The files that currently matter, and the concepts we have already used.

| FILE | ROLE |
| --- | --- |
| src/main.ts | Browser entry point. Calls bootstrapApplication(App, appConfig). |
| src/app/app.ts | Root standalone component. Its selector is app-root; it imports RouterOutlet. |
| src/app/app.html | Root template. router-outlet is the slot where the router displays the current page. |
| src/app/app.routes.ts | Maps the empty path / to HomePage. |
| src/app/features/home/home-page/* | HomePage TypeScript, HTML, and component CSS. CLI created a starter placeholder. |
| src/index.html | Angular document shell. The app-root element is where Angular mounts the root component. |
| src/app/app.config.ts | Registers router and client hydration providers for the browser app. |
| src/main.server.ts / src/server.ts | Server bootstrap and SSR server entry point. |
| src/styles.css / angular.json | Global CSS and Angular CLI build/serve/SSR configuration. |

## Important distinction

There are two index.html files. src/index.html is used by Angular and must keep the app-root element. The root-level index.html belongs to the older static prototype. We accidentally encountered the selector error when the Angular entry document lacked its host element; restoring it fixed the mount point.

## Standalone component

Each component declares the template dependencies it uses in its own imports array. Modern Angular creates standalone components by default, so this project has no AppModule. HomePage is a page-level component; sections and reusable UI can be extracted when the design needs them.

## Signal and template binding

The root component used signal('Vishnu'); the template reads the value with {{ title() }}. Changing the signal value and seeing the page update was the first live Angular edit. Signals are reactive state holders; interpolation places a value into the template.

## Routing

app.routes.ts maps a URL to a component. RouterOutlet is the placeholder inside the root template where the routed component appears. Route setup and the outlet are both needed: one chooses the page, the other gives Angular a place to render it.

# 03 / Rendering and the next architecture

A short mental model for CSR, prerendering, SSR, backend content, and admin security.

| MODE | WHEN HTML IS MADE | GOOD FIT |
|---|---|---|
| CSR | In the visitor's browser | Highly interactive app areas; simplest hosting model |
| Prerender / SSG | During the build | Stable public pages; CDN-friendly; content changes usually need a rebuild |
| SSR | On a server for each request | Fresh public content with SEO; requires a server runtime |

## What this project has

The CLI scaffold has server rendering and hydration configuration. Current app.routes.server.ts uses a prerender rule, but the app route list was empty until HomePage was wired. When backend-powered blog routes arrive, we will revisit route rendering. Prerendered dynamic slugs require build-time data; SSR can render fresh data on requests but needs a server host. Hydration makes server-rendered HTML interactive in the browser.

## A sensible growth path

- Finish the home page as a vertical slice: page shell, hero, experience, skills, work, About, and contact.
- Start with typed local sample content behind a content service; later replace its data source with HttpClient/API calls.
- Add blog list/detail routes after the home page is stable; decide per-route prerender vs SSR based on publishing cadence and hosting.
- Build the admin later. Angular Material is a deliberate learning objective for admin forms/tables.
- Protect admin data at the backend. Angular route guards are navigation controls, not security enforcement. Never expose secrets in browser code.

## Typical team practice

There is no single 'MNC way', but mature teams usually agree on user needs and route/data boundaries, use feature-oriented folders and consistent conventions, deliver a thin vertical slice, review changes, and verify accessibility, security, and build quality. The practical lesson is to create the next useful boundary, not every future folder up front.

## Angular packages already in this project

@angular/core provides components/signals; @angular/router provides routes; @angular/common includes HttpClient for future API calls; @angular/platform-browser and @angular/platform-server support browser/server rendering; @angular/ssr supplies SSR integration. Angular Material is not installed yet; we plan to add it when the admin UI is built.

## Current next milestone

**Add the actual home-page content incrementally, starting with the hero. Keep HomePage as the page orchestrator; extract a section component when it has a clear responsibility or reuse case.**

# 04 / Interview perspective: Angular basics

Answers should be truthful to the current project state. Personalize the wording and practise saying it aloud.

### 1. What are you building?

A personal portfolio for hiring managers, recruiters, and potential clients. It will present experience and skills, selected case studies, an About story, and contact details. I have a working Angular route and am building the page incrementally; backend, blog, and admin features are planned, not completed.

### 2. What is an Angular component?

A component is a UI building block made from a TypeScript class plus a template and optional styles. The class holds the view's data and behavior; the template describes its HTML and Angular bindings.

### 3. What does standalone mean?

A standalone component can be used directly without being declared in an NgModule. Its imports array lists the components, directives, and pipes its template needs. Modern Angular CLI components are standalone by default.

### 4. How is HomePage connected to the URL?

The route configuration maps the empty path to HomePage. The root template contains RouterOutlet, and Angular renders the route's component there. The route selects the component; the outlet is its render location.

### 5. What is a signal?

A signal is a reactive value container. A template can read it, and Angular tracks that dependency so the view updates when the signal changes. I used a title signal and interpolation as my first experiment.

### 6. Why use feature-oriented folders?

Related code stays near the feature it supports. It makes boundaries and ownership easier to understand than putting every component or service into a large type-based folder. I created the home page under features/home.

### 7. What is SSR?

Server-side rendering produces the initial HTML on a server for a request, then the browser hydrates it so Angular can make it interactive. It can help content visibility and SEO, but it requires server-compatible code and hosting.

# 05 / Interview perspective: decisions and tradeoffs

Avoid claiming future work as if it is already implemented.

### 8. How does prerendering differ from SSR?

Prerendering creates HTML at build time, which is fast to serve and suits stable public content; publishing changes normally requires a rebuild. SSR creates HTML per request, which can show fresher content but needs a server runtime.

### 9. How will portfolio content become dynamic?

I plan to begin with typed local sample content behind a content service. Later the service can use Angular HttpClient to call a versioned backend API while the page components keep consuming stable view models. That integration is future work.

### 10. How will the admin be secured?

The backend will authenticate users and authorize every protected action. Angular guards can control navigation but cannot secure an API. I would keep secrets on the server and use a reviewed session/authentication design.

### 11. Why Angular Material for the admin?

It is a deliberate learning goal and can provide consistent accessible form controls, dialogs, menus, and tables for the admin interface. The public portfolio can keep its custom visual design; Material is not currently installed.

### 12. How did you resolve NG05104?

Angular was bootstrapping the root component with selector app-root, but src/index.html did not contain the app-root host element. Restoring that element fixed the mount point. I also corrected the router-outlet tag in the root component template.

### 13. What is the difference between src/index.html and the root index.html?

src/index.html is Angular's entry document and contains app-root. The root-level index.html belongs to the older static prototype. Angular serves/builds from the src entry configured in angular.json.

### 14. What is implemented now, and what remains?

The Angular CLI scaffold, root route, HomePage component, router outlet, and localhost rendering are working. The full portfolio UI, API, blog, admin, and deployment are not implemented yet. Git was initialized; I should confirm whether the baseline commit exists.

## A compact project introduction

**I am building a personal portfolio with Angular 22 using standalone components and the CLI's SSR-capable scaffold. I created a HomePage component, connected it to the root route through RouterOutlet, and got it rendering locally. My next step is to build the home page with typed content; blog/API/admin work is planned for later, with Angular Material reserved for the admin learning milestone.**

## Interview reminder

Explain what you did, why you chose it, and what you learned from the error or tradeoff. Do not describe SSR, API integration, accessibility, security, tests, or deployment as finished until you have implemented and verified them.

# 06 / Practice to refresh and learn

A sequence that reinforces fundamentals before introducing backend or admin complexity.

| STEP | PRACTICE | YOU SHOULD BE ABLE TO EXPLAIN |
|---|---|---|
| 1 | Open app.ts, app.html, HomePage files; identify class/template/styles. Change a signal and observe the template. | Component metadata, interpolation, signal reads, live reload. |
| 2 | Add a semantic hero section to HomePage; use headings, intro, portrait slot, and CTA. | Template structure, accessibility, component CSS scope. |
| 3 | Make the hero responsive; test narrow and wide viewports. Check keyboard focus and reduced motion. | CSS layout, breakpoints, accessible interactions. |
| 4 | Add experience and skills with typed local data and Angular @for; track each repeated item by stable id. | TypeScript interfaces, template control flow, stable lists. |
| 5 | Extract a project card only after deciding what data it accepts. Reuse it for multiple cards. | Inputs, standalone imports, component boundaries. |
| 6 | Move sample data behind a content service; later replace local data with HttpClient and explicit loading/error states. | Dependency injection, services, async data flow. |
| 7 | Add blog list/detail routes; choose route rendering based on how content is published. | Lazy routes, dynamic route params, SSR vs prerender tradeoffs. |
| 8 | Build protected admin API first; then add an Angular Material admin interface. | Backend authorization, forms, tables, theme/customization. |

## Useful commands

**npm start - run the local development server**
**npm run build - create a production build**
**ng generate component features/home/section-name --standalone --skip-tests - generate a component with colocated files**
**git status - review changes before a commit**

## Short active-recall quiz

- Without looking: explain the path from browser URL / to HomePage.
- Point to the element Angular mounts into and the element where the router renders a page.
- Name one benefit and one cost of prerendering and SSR.
- Explain why a future Angular admin guard is not enough to secure backend data.

## Suggested review rhythm

After each development slice, write three short notes: what changed, what Angular concept it used, and what you would verify next. Revisit the quiz before interviews and update this guide when the implementation changes.

# 07 / Quick reference

A compact file map and trusted resources for follow-up study.

## The route-to-page chain

**src/index.html provides <app-root> → src/main.ts bootstraps App → App template provides <router-outlet> → app.routes.ts maps / to HomePage → HomePage template renders its view.**

## Angular references

Component guide - standalone components and component templates

Signals guide - reactive state

Template control flow - @if and @for

Routing guide - routes and outlets

Rendering strategies - CSR, prerender, SSR

Angular style guide - project conventions

## Project documents

README.md, docs/architecture.md, docs/content-and-api.md, docs/content-authoring.md, docs/development-workflow.md, docs/production-readiness.md, docs/roadmap.md, and docs/decisions/0001-angular-and-rendering.md hold the project's current written guidance. Update them when decisions change.

## Version note

The project snapshot used Angular CLI 22.2.2, Angular core/router 22.2.x, Node 24.21.0, npm 11.14.1, and Git for Windows 2.56.0. Recheck Angular's official version compatibility page before upgrading or changing the Node runtime.

END OF CURRENT STUDY GUIDE / UPDATE AS THE PROJECT GROWS